



软件架构设计

主讲：李伟刚

liweigang@nwpu.edu.cn

西北工业大学软件学院



第七章 客户端/服务器软件架构设计

主要内容

- 7.1 概述
- 7.2 客户端/服务软件架构的结构模式
- 7.3 客户端/服务软件架构的通信模式
- 7.4 客户端/服务器系统的中间件
- 7.5 服务子系统的设计
- 7.6 包装器类的设计
- 7.7 从静态模型到关系数据库设计



7.1 概述

- 客户端是服务的请求者
- 服务器是服务的提供者
 - **服务器**是一个为多个客户端提供一个或多个服务的硬件/软件系统
 - **服务**在**C/S**系统中是指一个满足多个客户端需要的应用软件构件
 - 服务器可以支持单一服务，也可以支持多种服务
 - 一个大的服务可以跨越多个服务器结点
- **C/S**架构基于客户端/服务架构模式



7.2 客户端/服务软件架构的结构模式

- 包括：

- 多客户端/单服务架构模式
- 多客户端/多服务架构模式
- 多层客户端/服务架构模式



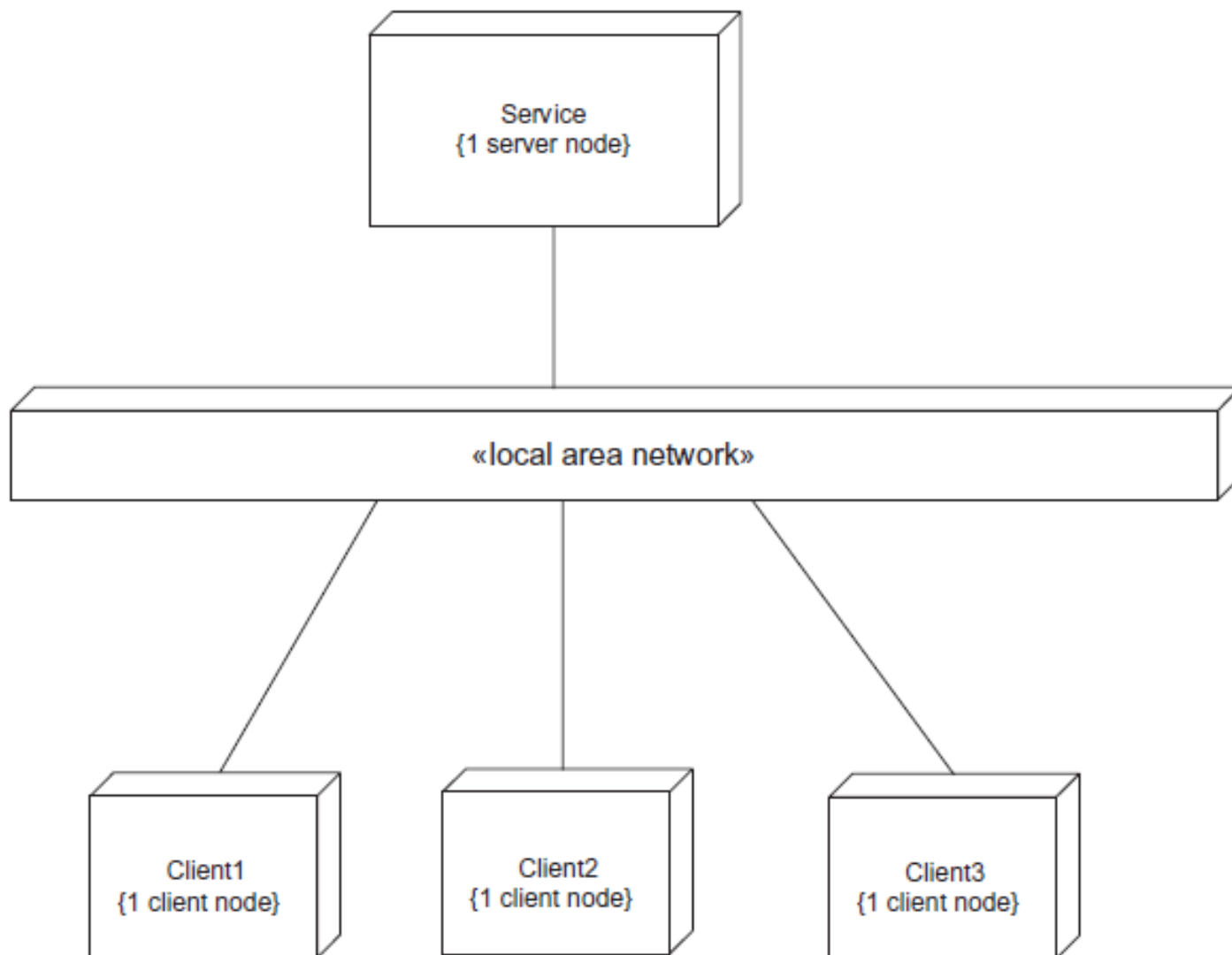
7.2 客户端/服务软件架构的结构模式

- 多客户端/单服务架构模式

- 是客户端/服务器（**C/S**）架构的最常见模式，因此常直接称它为**C/S模式**
- 一般采用顺序性通信方式
- 部署图



7.2 客户端/服务软件架构的结构模式



7.2 客户端/服务软件架构的结构模式

- 多客户端/多服务架构模式

- 多对多通信

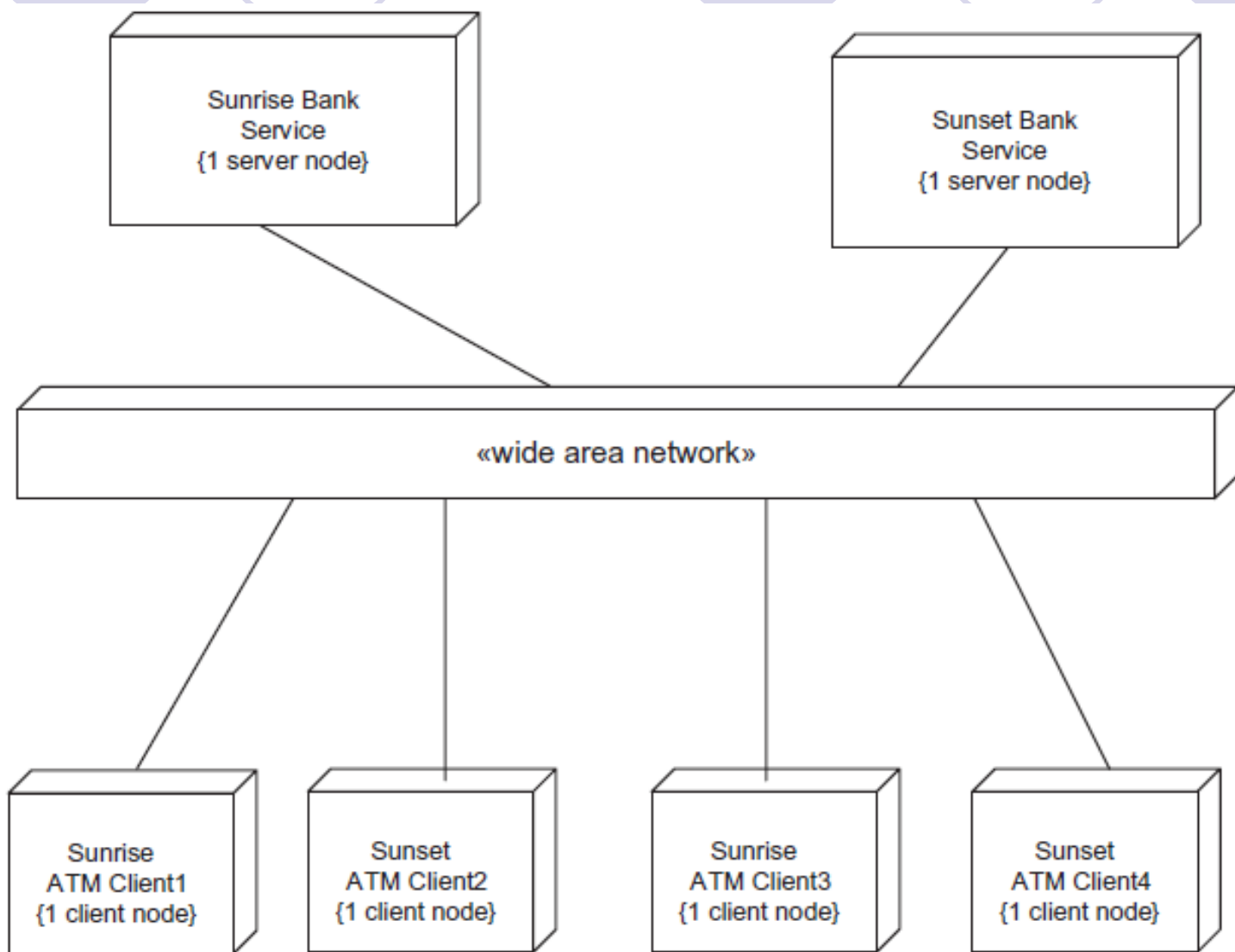
- 服务之间也可能相互通信

- 既可以采取顺序性通信，也可以采用并发通信

- 例子：银行联盟

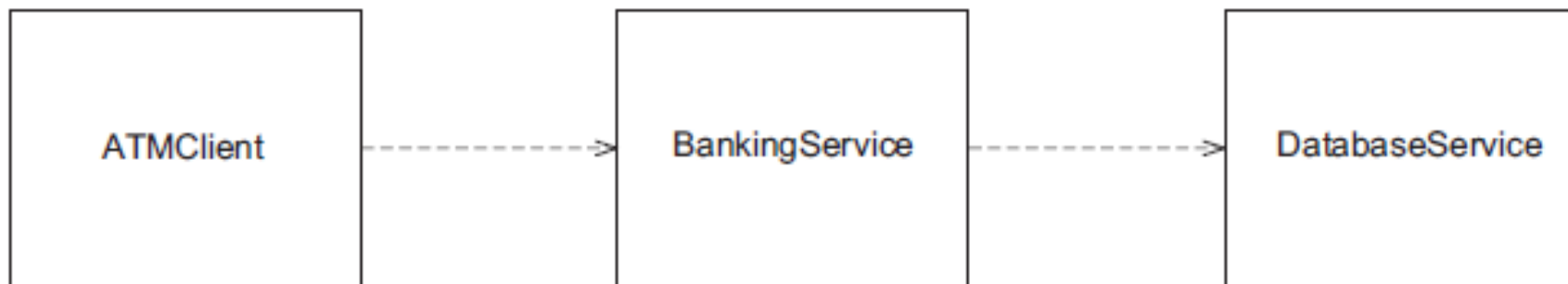


7.2 客户端/服务软件架构的结构模式



7.2 客户端/服务软件架构的结构模式

- 多层客户端/服务架构模式
 - 拥有同时扮演着客户端和服务角色的中间层
 - 中间层可能有多个
 - 具有服务功能的层可部署在同一个服务器结点上
 - 例子：一个三层的银行系统



7.3 客户端/服务软件架构的通信模式

- 客户端向服务发送请求并可能从服务接到一个回复
- 客户端和服务之间的通信状况影响着通信模式的选择
- 通信模式包括
 - 带回复的同步消息通信
 - 异步消息通信
 - 带回调的异步消息通信
 - 不带回复的同步通信
 - 代理者模式
 - 群组通信模式



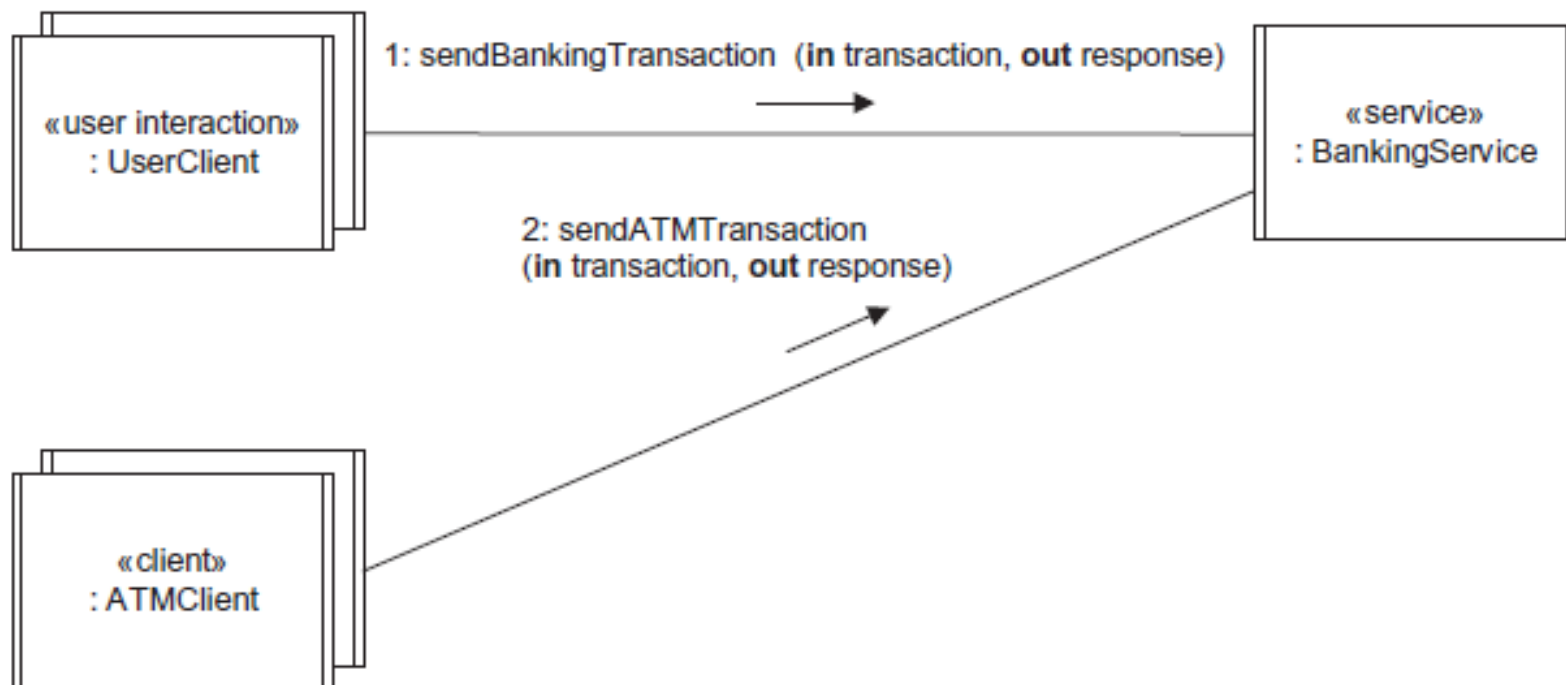
7.3 客户端/服务软件架构的通信模式

- 带回复的同步消息通信模式
 - 是**C/S**模式中最常用的软件架构通信模式
 - 也称作“**请求/响应**”模式
 - 客户端向服务发送消息并等待回复
 - 由于客户端可能有多个，所以客户端发送到服务的请求需要在服务端的消息队列中排队
 - 服务以先来先服务（**FIFO**）的原则处理收到的消息，并发送同步的回复消息给相关客户端
 - 若客户端和服务端之间存在包含多条消息和回复的会话，则它们之间会建立一个连接



7.3 客户端/服务软件架构的通信模式

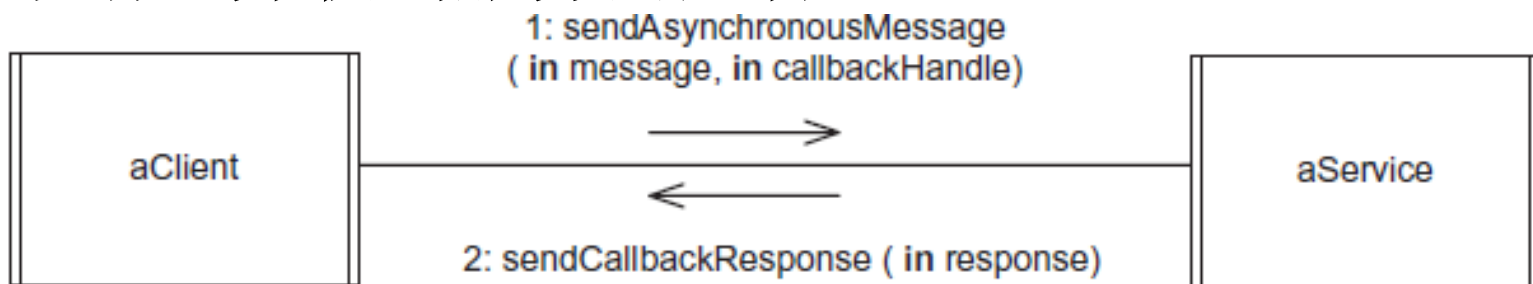
- 例子：银行应用



7.3 客户端/服务软件架构的通信模式

- 带回调的异步消息通信模式

- 客户端向服务发送了一条请求后可以继续工作而不必等待应答
- 服务需要在**稍后**向客户端发送答复信息（异步应答）
- 一个客户端在同一时间只会向服务发送一条请求消息并收到服务的应答



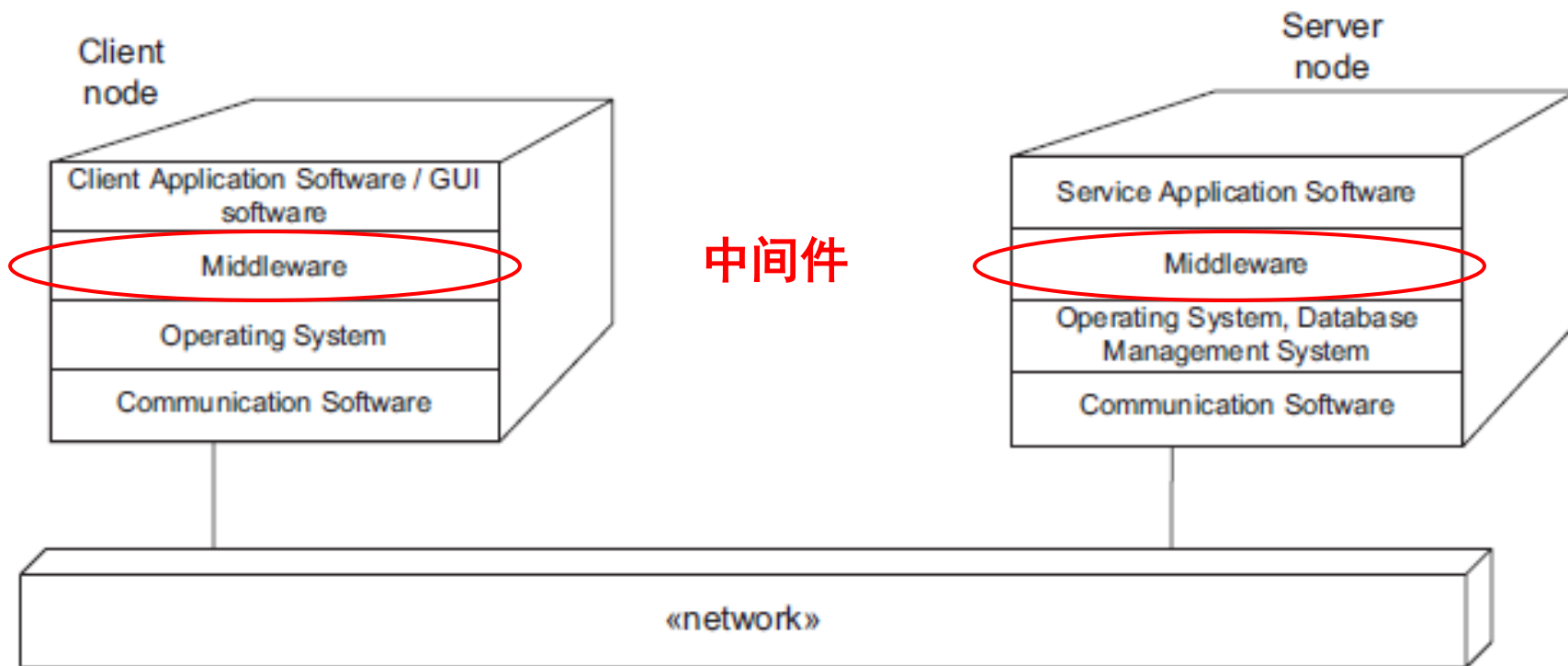
7.4 客户端/服务器系统的中间件

- **C/S**架构中的通信经常是同步的，通常是由中间件技术来提供支持
- 远程过程调用（**RPC**）
 - 是一种早期的**C/S**同步通信中间件技术
 - 基于**RPC**的中间件有
 - 分布式计算环境（**DCE**）
 - **Java**远程方法调用（**RMI**）
 - 构件对象模型（**COM**）
 - 通用对象请求代理体系结构（**CORBA**）



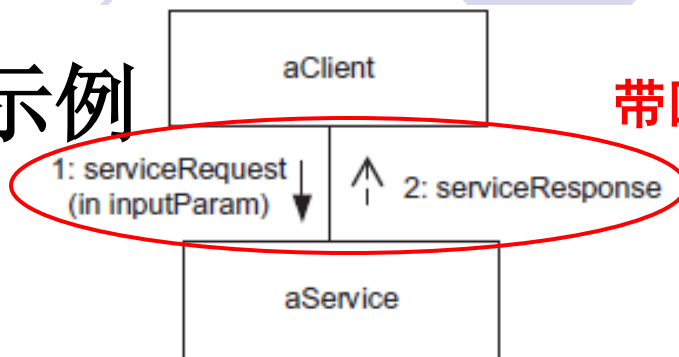
7.4 客户端/服务器系统的中间件

● 基于中间件的C/S通信示例

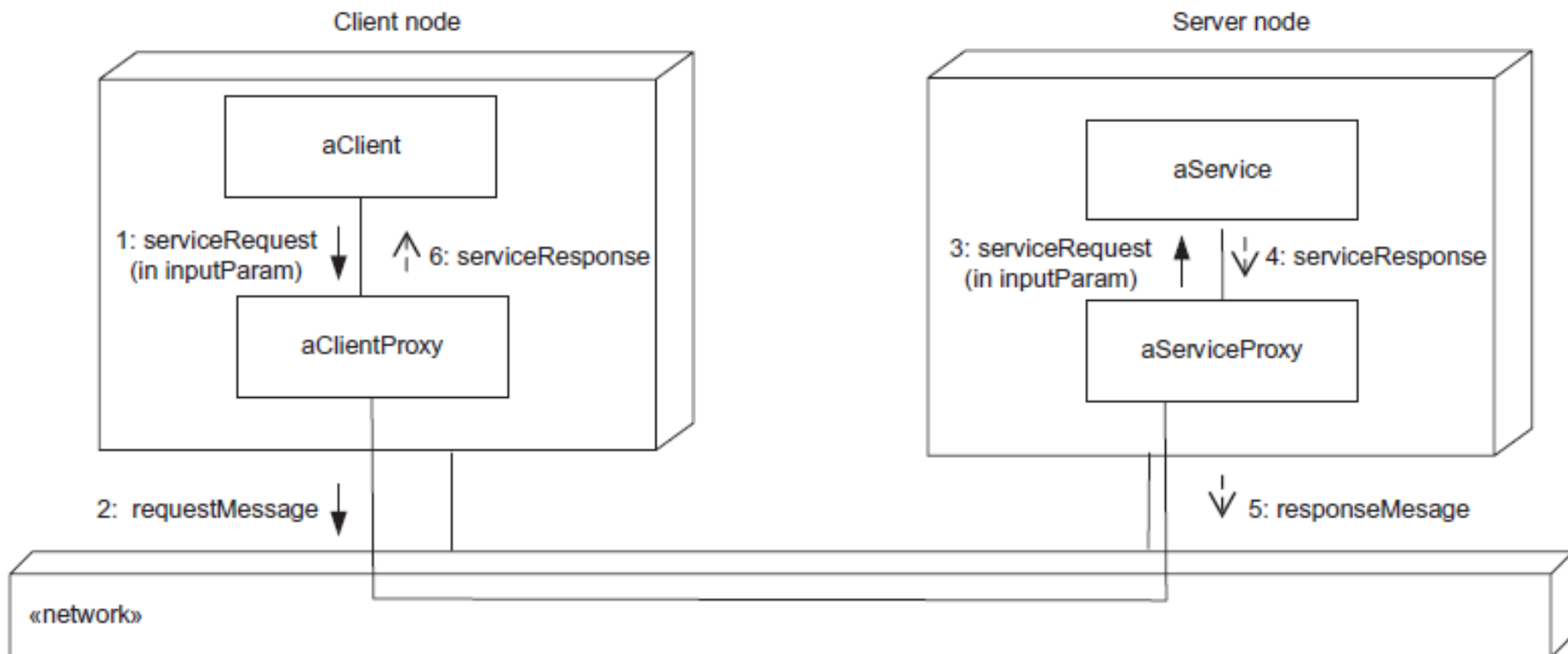


7.4 客户端/服务器系统的中间件

● Java RMI 示例



带回复的同步消息通信



7.5 服务子系统的设计

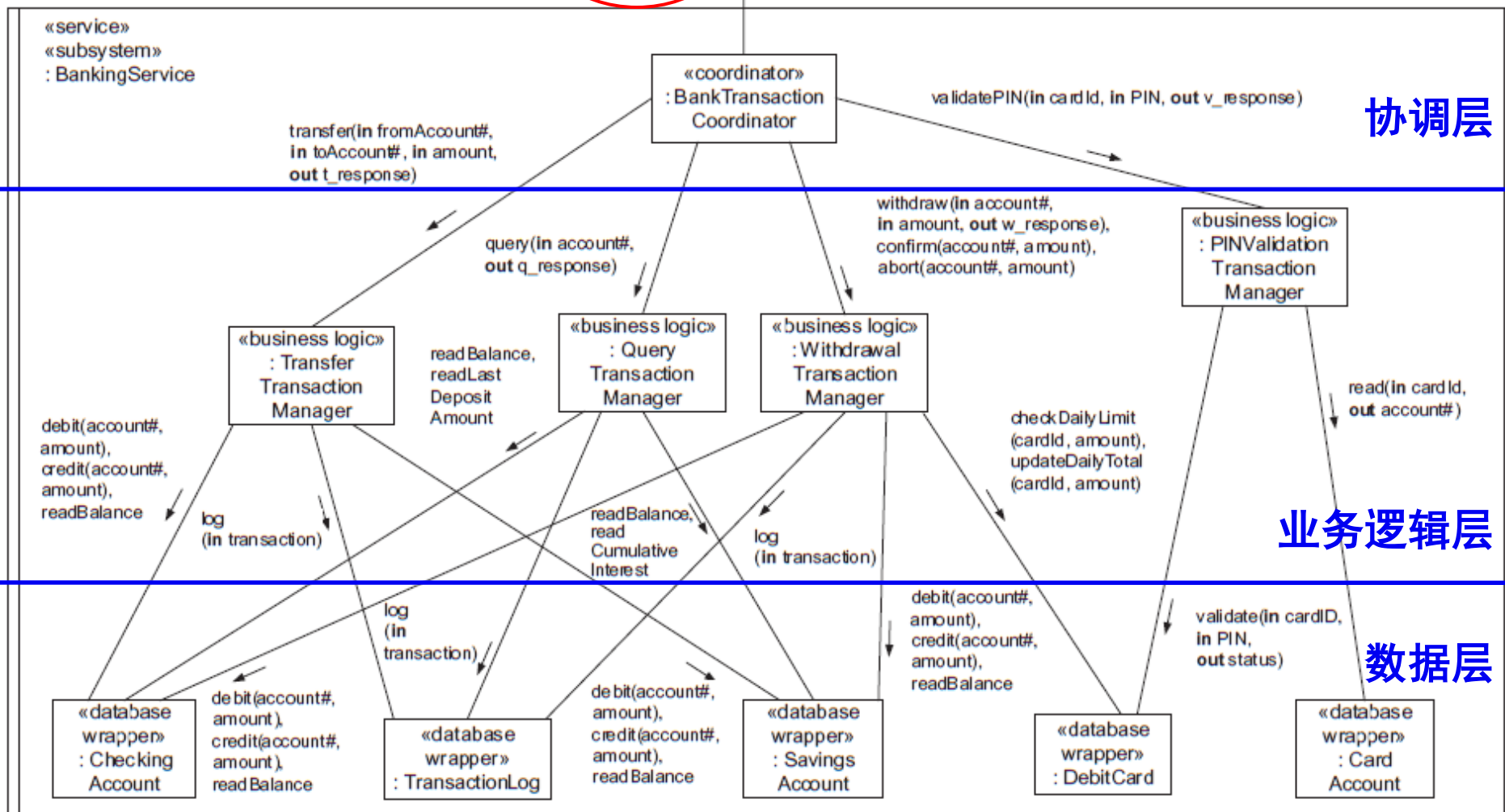
● 顺序性服务的设计

- 服务必须先完成上一个服务请求，才能开始处理下一个请求
- **顺序性服务**设计为一个并发对象（控制线程）来响应客户端的服务请求
- 服务通常会有一个消息队列来存放收到的服务请求
- 服务协调者收到客户端消息后，按照消息类型调用相应服务对象所提供的操作
- 消息的参数作为操作的参数
- 服务对象的处理结果经由服务协调者发回客户端



带回复的同步消息通信

对象之间的所有通信都是通过操作调用的方法



顺序性服务设计只在服务器结点有足够的
能力充分应对交易请求频率的情况下使用



西北工业大学

NORTHWESTERN POLYTECHNICAL UNIVERSITY

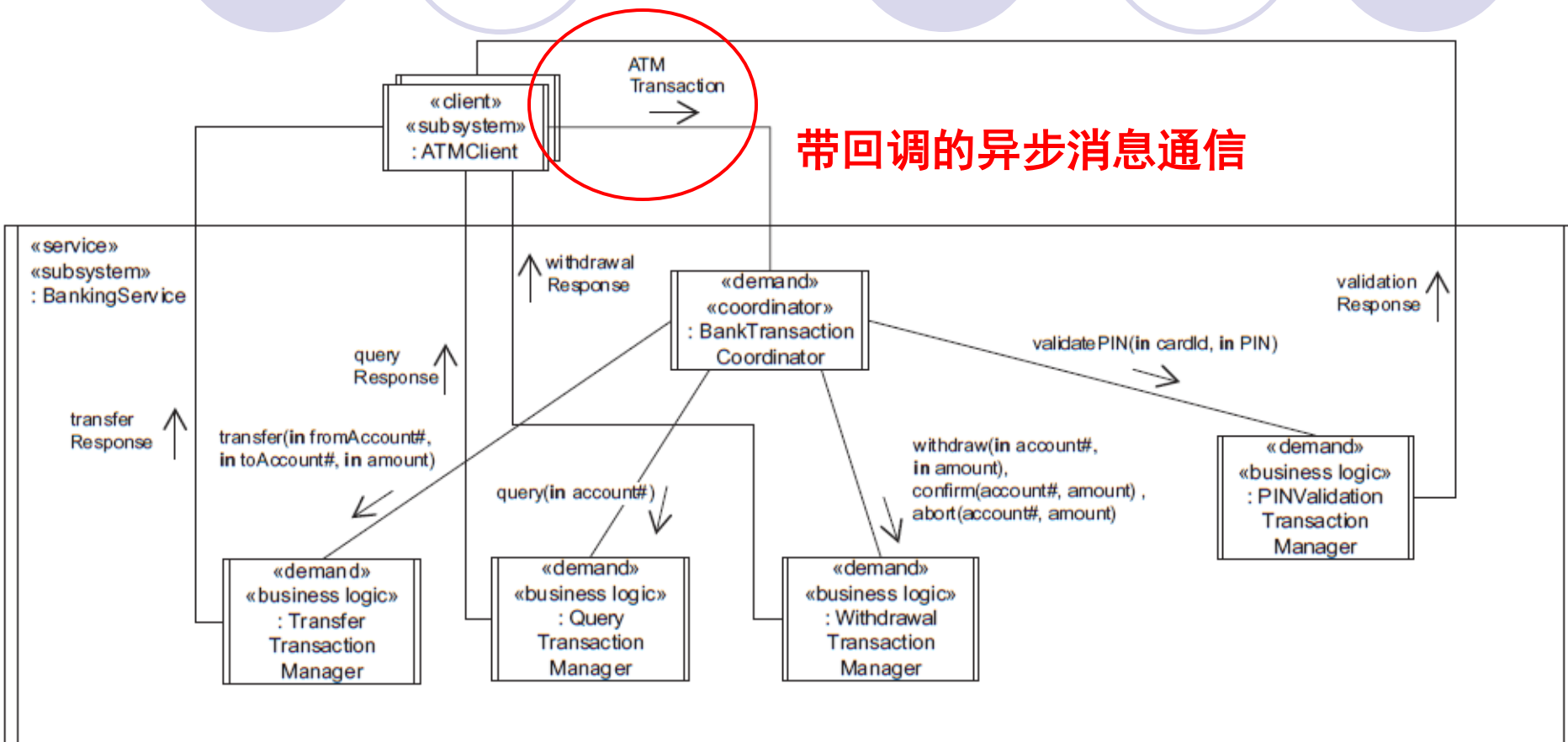
7.5 服务子系统的设计

- 并发服务设计

- 服务功能由多个并发的对象共同完成
- 适用于客户端对于服务的要求很高，使用多个并发对象提供的服务可以提高系统的吞吐量
- 使用并发服务设计的例子：“银行服务”的另一种设计方案



7.5 服务子系统的设计



7.6 包装器类的设计

- 包装器类是一种处理客户端对于遗产应用请求的通信和管理的服务器类
- 包装器将来自客户端的外部请求转化为对遗产代码的调用，同时将遗产代码的输出转化成对客户端的相应
- 包装器类也可能通过更松散的机制实现与遗产应用的接口，如中间文件
- 包装器可以封装对数据库的访问



7.6 包装器类的设计

● 数据库包装器类的设计

- 分析模型中的实体类在设计阶段需要重新考虑
 - 其封装的数据由实体类直接管理 → **数据抽象类**
 - **封装了数据结构**
 - 或存储在数据库中 → **数据库包装器类**
 - **隐藏了数据是如何被访问的**
- C/S模式中，数据抽象类更常位于客户端，数据库包装器类一般位于服务器端
- 数据库包装器类可以提供面向对象的数据库访问接口
- 需要确定实体类中有哪些需要映射到关系数据库并将它们设计为数据库包装器类



7.6 包装器类的设计

- 有时，通过数据库包装器类从数据库中检索到的数据会被临时保存在数据抽象类中
- 数据库包装器类隐藏了如何访问保存在关系表中的数据的细节
- 示例：“银行系统”中的实体类“借记卡”



7.6 包装器类的设计

分析模型

«entity» DebitCard
cardID : String PIN : String startDate : Date expirationDate : Date status : Integer limit : Real total : Real

设计模型

«database wrapper» DebitCard
+ create (cardID) + validate (in cardID, in PIN, out status) + updatePIN (cardID, PIN) + checkDailyLimit (cardID, amount) + updateDailyTotal (cardID, amount) + updateExpirationDate (cardID, expirationDate) + updateCardStatus (cardID, status) + updateDailyLimit (cardID, newLimit) + clearTotal (cardID) + read (in cardID, out PIN, out expirationDate, out status, out limit, out total) + delete (cardID)



7.7 从静态模型到关系数据库设计

- 一般，每个实体类都会被设计为一张关系数据库表
- 实体类的每个属性对应关系表的某列
- 每个实体对象（类实例）对应表中一行数据
- 关系数据库表必须有一个主键，一般能够唯一确定表中某一行的属性选做主键，可以是复合主键
- 简单的关联关系（一对一，一对多）可以用外键表示



7.7 从静态模型到关系数据库设计

- 外键是包含在一张表中的另一张表的主键，可以帮助从一张表导航到另一张表

○ 例如：客户-账户

Customer
customerName: String customerId: String customerAddress: String

1



Owns

1..*

Account
accountNumber: Integer balance: Real

将“Customer”表的主键“customerId”
作为外键添加到“Account”表中

<u>accountNumber</u>	Balance	customerId
1234	398.07	24193
5678	439.72	26537
1287	851.65	21849

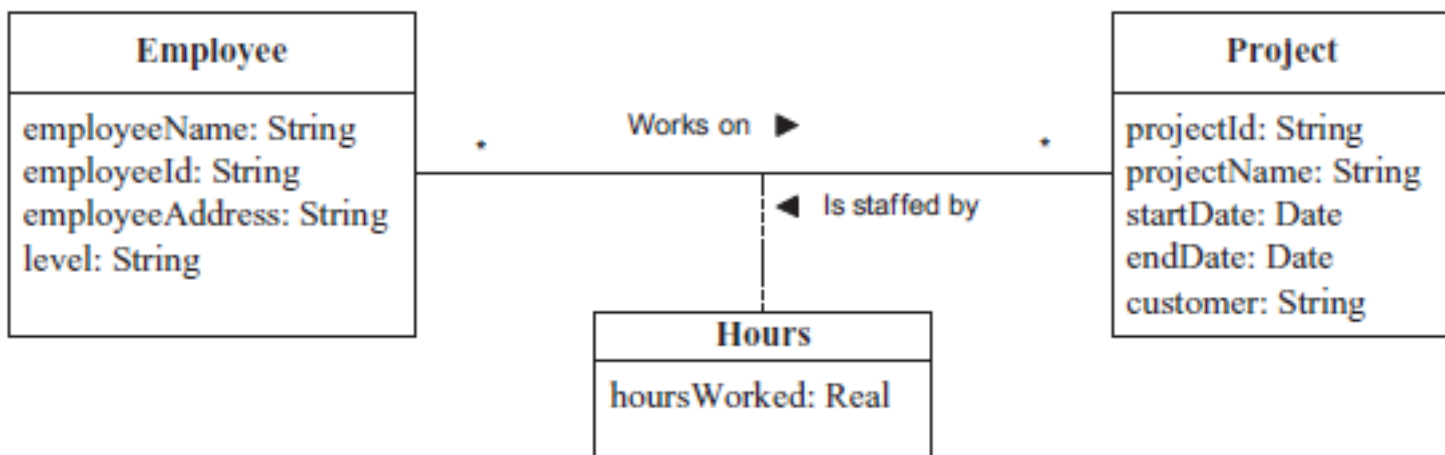
<u>customerName</u>	<u>customerId</u>	customerAddress
Smith	21849	New York
Patel	26537	Chicago
Chang	24193	Washington



7.7 从静态模型到关系数据库设计

- 多对多关联一般用关联类表示
- 关联类被映射为 **关联表**
- 关联表的主键是一个复合键，由参与关联的关系表的主键组成

○ 例如



Project (projectId, projectName)

Employee (employeeId, employeeName, employeeAddress)

Hours (projectId, employeeId, hoursWorked)



7.7 从静态模型到关系数据库设计

- 将整体/部分关系映射到关系数据库
 - 聚合或组合类（整体部分）和每个部分类都被设计成一张关系表
 - 整体类和部分类的主外键关系参考前述一对一、一对多关联



7.7 从静态模型到关系数据库设计

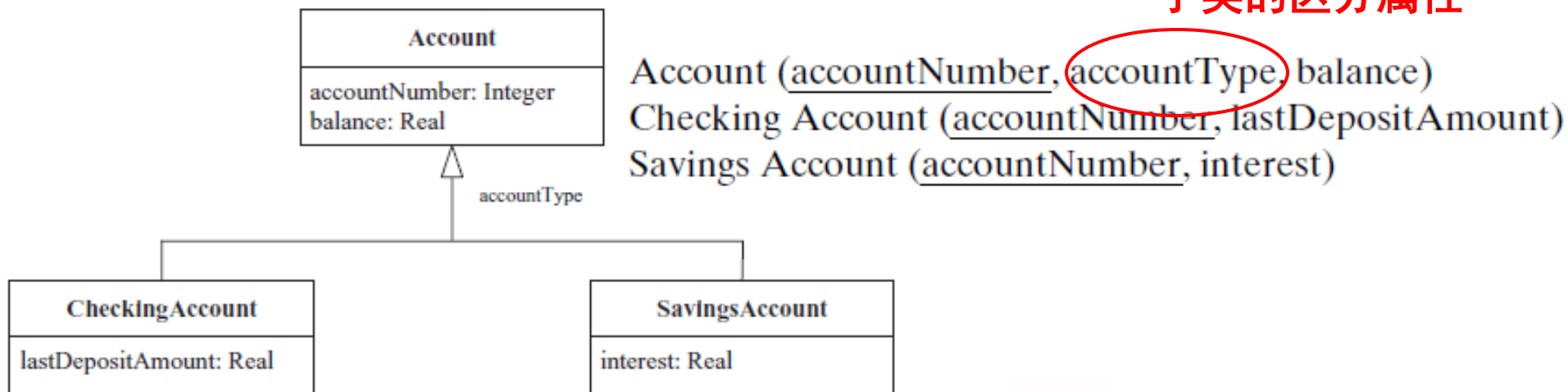
● 将泛化/特化关系映射到关系数据库

○ 三种映射方法

● 每个父类和子类都分别映射为一张关系表

- 父类子类共享主键
- 优点：简洁、可扩展
- 缺点：父/子类间导航效率低，在父类对应的关系表中需要定义子类的区分属性

子类的区分属性



7.7 从静态模型到关系数据库设计

- 只将子类映射为关系表
 - 每个子类都会对应一张表，没有对应于父类的表
 - 父类的属性在子类的表中重复出现
 - 若父类属性少时较适合
- 只将父类映射为关系表
 - 所有子类属性都放到父类对应的表中
 - 父类中需要有区分子类的属性
 - 父类中每一行都会描述某个子类的属性，与该子类无关的属性被设为空值
 - 表设计冗余



